

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №4
по дисциплине «Методы тестирования программного
обеспечения»
на тему «Автоматизированное тестирование»
Вариант 2

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Курочкин М. А. _____

« ____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задач	4
2 Описание средств автоматизации тестирования	5
2.1 JUnit	5
2.1.1 Особенности JUnit	5
2.1.2 Этапы написания тестов	5
2.2 Selenium WebDriver	6
2.2.1 Особенности Selenium WebDriver	7
2.2.2 Основные методы WebDriver	7
2.2.3 Этапы работы с WebDriver	9
2.3 TestNG	9
3 Задача 1. Юнит-тестирование библиотеки	10
3.1 Настройка проекта	10
3.2 Тестирование метода isLeapYear	11
3.3 Тестирование метода getDaysInMonth	12
3.4 Тестирование метода isValidDate	13
3.5 Тестирование метода addDays	14
3.6 Результаты юнит-тестирования	16
4 Задача 2. UI-тестирование веб-сайта	17
4.1 Тестовые сценарии	17
4.2 Настройка драйвера (класс DriverSetup)	19
4.3 Класс NavigationTrainigResult	20
4.3.1 Навигация к целевой странице	20
4.3.2 Выполняемые проверки	20
4.3.3 Отладка	21
4.4 Класс HomePageNavigationTestsResult	21
4.4.1 Выполняемые проверки	21
4.5 Результаты тестирования	23
Заключение	24
Список использованных источников	25

Введение

Под *тестированием программного обеспечения* понимается процесс выявления расхождений между реальным поведением программы и тем, что указано в её спецификации.

Автоматизированное тестирование представляет собой способ верификации ПО с помощью специальных инструментов и фреймворков, которые позволяют выполнять однотипные проверки без участия человека. К числу его достоинств можно отнести более предсказуемые результаты по сравнению с ручным тестированием, поскольку исключается влияние человеческого фактора. За счёт широкого охвата сценариев автоматизация способствует поддержанию стабильного качества и производительности программного продукта.

1 Постановка задач

Цель работы Изучить и применить инструменты автоматизированного тестирования: фреймворк JUnit для модульного-тестирования и Selenium WebDriver вместе с TestNG для UI-тестирования веб-приложений.

Задачи

1. Создать unit-тесты для библиотеки `lab-1.0.jar` с использованием фреймворка JUnit.
 - (a) Разработать unit-тесты для методов класса `DateUtils.java`.
 - (b) Запустить тесты при помощи используемой среды разработки.
 - (c) Продемонстрировать результаты выполнения тестов и проанализировать полученные дефекты.
2. Провести тестирование веб-сайта с использованием TestNG и Selenium WebDriver.
 - (a) Написать два теста, проверяющих корректное отображение и работу страниц веб-сайта.
 - (b) Организовать код тестов в соответствие с Java Code Convention.
 - (c) Запустить тесты через конфигурационный файл TestNG suite xml.

2 Описание средств автоматизации тестирования

2.1 JUnit

JUnit – фреймворк для модульного тестирования программного обеспечения, разработанного на языке программирования Java. Он позволяет разработчикам создавать и выполнять автоматизированные тесты, чтобы убедиться, что код работает корректно, а также обнаруживать ошибки на ранних этапах разработки.

2.1.1 Особенности JUnit

- **Аннотации.** JUnit предоставляет ряд аннотаций для задания и настройки текстов.
- **Поддержка параметризованных тестов.** Использование аннотации `@ParametrizedTest` позволяет прогнать один и тот же тест с разными параметрами.
- **Утверждения (assertions).** Для проверки ожидаемых результатов JUnit предоставляет статические методы класса `Assertions`. Поддержка групповых проверок через `assertAll()` позволяет выполнять все переданные утверждения независимо от результатов предыдущих.
- **Интеграция с различными средами разработки.** JUnit интегрируется с большинством IDE, в том числе с IntelliJ IDEA и Eclipse.

2.1.2 Этапы написания тестов

1. **Создание тестов.** Написание методов и добавление к ним аннотации `@Test` – эта аннотация указывает, что метод является тестовым.
2. **Настройка тестов.** Настройка теста происходит с использованием предоставляемых JUnit аннотаций:
 - (a) `@BeforeEach`: метод, помеченный данной аннотацией, выполняется перед *каждым* тестовым методом в классе. Используется для инициализации объектов и подготовки данных, требуемых для каждого отдельного теста.
 - (b) `@AfterEach`: метод выполняется после *каждого* тестового метода. Предназначен для очистки ресурсов, сброса состояния или удаления временных данных, созданных в ходе выполнения теста.

- (c) **@BeforeAll**: метод выполняется однократно до выполнения *всех* тестовых методов в классе. Требует, чтобы метод был статическим (`static`). Применяется для ресурсоёмких операций инициализации, таких как запуск сервера, открытие соединения с базой данных или создание экземпляра веб-драйвера.
- (d) **@AfterAll**: метод выполняется однократно после выполнения *всех* тестовых методов. Также требует статического объявления. Используется для финального освобождения ресурсов: закрытия соединений, остановки сервисов или удаления глобальных временных файлов.
- (e) **@Disabled**: аннотация для временного отключения тестового метода или целого класса. Такой тест не выполняется, а в отчётах помечается как пропущенный (`skipped`). Позволяет исключить из прогона некорректные или неактуальные тесты без удаления исходного кода.
- (f) **@Nested**: позволяет создавать вложенные классы внутри основного тестового класса. Каждый вложенный класс формирует отдельную группу тестов с собственным жизненным циклом (собственные методы **@BeforeEach** и **@AfterEach**). Удобен для логической группировки тестов по сценариям или состояниям системы.
- (g) **@Tag**: служит для маркировки тестов произвольными метками. При запуске тестов можно указать, какие теги включать или исключать, что позволяет формировать набор выполняемых тестов без изменения кода.
- (h) **@TestFactory**: указывает, что метод является фабрикой для создания динамических тестов. В отличие от статического теста (**@Test**), такой метод в процессе выполнения генерирует один или несколько объектов `DynamicTest`. Применяется, когда набор тестовых случаев заранее неизвестен и формируется на основе внешних данных.

3. **Запуск тестов.** Выполнение тестов с помощью инструментов IDE или командной строки.

2.2 Selenium WebDriver

Selenium WebDriver представляет собой библиотеку и фреймворк для автоматизации действий веб-браузера. Он предоставляет программный интерфейс, позволяющий управлять браузером так, как это делает реальный пользователь: открывать страницы, заполнять формы, кликать по элементам, перемещаться между страницами. WebDriver взаимодействует с нативными средствами автоматизации конкретного браузера (ChromeDriver, GeckoDriver

для Firefox, EdgeDriver и др.), что обеспечивает максимально реалистичное поведение.

2.2.1 Особенности Selenium WebDriver

- **Кросс-браузерность.** WebDriver поддерживает все основные браузеры: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Opera. Один и тот же тестовый сценарий может быть выполнен на разных браузерах без изменения кода.
- **Поддержка нескольких языков программирования.** Фреймворк предоставляет реализации для Java, C#, Python, Ruby и Kotlin.
- **Ожидания (waits).** Для борьбы с асинхронностью веб-приложений (подгрузка данных, анимация, динамическое появление элементов) WebDriver предлагает три типа ожиданий: явные (explicit wait), неявные (implicit wait) и фиксированные (fluent wait).
- **Эмуляция действий пользователя.** Класс `Actions` предоставляет возможность имитировать сложные сценарии взаимодействия: наведение курсора (hover), перетаскивание (drag-and-drop), двойной клик, контекстное меню (правый клик), нажатие клавиш-модификаторов (Ctrl, Shift, Alt) в комбинации с кликом.
- **Работа с окнами и вкладками.** WebDriver позволяет переключаться между разными окнами и вкладками браузера, получать их идентификаторы, закрывать текущее окно или переходить на новое.

2.2.2 Основные методы WebDriver

Ниже приведены ключевые методы интерфейса `WebDriver` и вспомогательных классов, наиболее часто используемые при написании тестов:

- **Навигация:**
 - `get(String url)` — открывает указанный URL в текущем окне браузера,
 - `navigate().back()` — эмулирует нажатие кнопки «Назад» в браузере,
 - `navigate().forward()` — эмулирует нажатие кнопки «Вперёд»,
 - `navigate().refresh()` — перезагружает текущую страницу;
- **Управление окном:**

- `manage().window().maximize()` — разворачивает окно браузера на весь экран,
- `manage().window().setSize(Dimension size)` — устанавливает заданный размер окна,
- `getWindowHandle()` — возвращает идентификатор текущего окна,
- `getWindowHandles()` — возвращает набор идентификаторов всех открытых окон/вкладок,
- `switchTo().window(String handle)` — переключает контекст на указанное окно;

- **Поиск элементов:**

- `findElement(By locator)` — возвращает первый найденный элемент, соответствующий локатору,
- `findElements(By locator)` — возвращает список всех элементов, соответствующих локатору;

- **Методы интерфейса WebElement:**

- `click()` — выполняет клик по элементу,
- `sendKeys(CharSequence... keys)` — вводит текст в поле ввода или отправляет нажатия клавиш,
- `clear()` — очищает содержимое поля ввода,
- `getText()` — возвращает видимый текст элемента,
- `isDisplayed()` — проверяет, видим ли элемент на странице,
- `isEnabled()` — проверяет, доступен ли элемент для взаимодействия,
- `isSelected()` — проверяет, выбран ли элемент,
- `submit()` — отправляет форму (аналог нажатия Enter внутри формы);

- **Дополнительные возможности:**

- `executeScript(String script, Object... args)` — выполнение JavaScript-кода,
- `getCurrentUrl()` — возвращает URL текущей страницы,
- `getTitle()` — возвращает заголовок текущей страницы,
- `quit()` — закрывает все окна браузера и завершает сессию драйвера,
- `close()` — закрывает текущее окно (вкладку), оставляя остальные открытыми.

2.2.3 Этапы работы с WebDriver

1. **Инициализация драйвера.** Создаётся экземпляр класса конкретного драйвера (например. Полученный объект реализует интерфейс `WebDriver` и служит основным инструментом управления браузером.
2. **Навигация к целевому веб-приложению.** С помощью метода `get()` или `navigate().to()` драйвер переходит по URL тестируемого приложения.
3. **Поиск элементов на странице.** Используя метод `findElement()` с соответствующим локатором, тест получает ссылку на веб-элемент.
4. **Выполнение действий с элементами.** Над найденными элементами вызываются методы, имитирующие пользовательское взаимодействие.
5. **Верификация состояния.** После выполнения действий тест проверяет ожидаемый результат.
6. **Обработка исключительных ситуаций.** При ошибках (например, элемент не найден, клик не сработал) `WebDriver` выбрасывает исключения. В коде теста эти ситуации могут быть обработаны с помощью блоков `try-catch` или через механизм повторных попыток (`retry`).
7. **Освобождение ресурсов.** После завершения всех тестов (или после каждого теста) вызывается метод `quit()`, который закрывает все окна браузера и завершает процесс драйвера.

2.3 TestNG

TestNG (Test Next Generation) представляет собой фреймворк для модульного, интеграционного и сквозного тестирования на языке Java. Он был создан как расширение возможностей JUnit, устраняя ряд их ограничений, в частности, в области конфигурации тестов, управления зависимостями и поддержки параллельного выполнения.

TestNG поддерживает параллельное выполнение тестов и интегрируется с другими инструментами и системами (Kenkins, Eclipse, Maven), что упрощает процесс тестирования и развертывания ПО.

3 Задача 1. Юнит-тестирование библиотеки

Дано: библиотечный файл `lab-1.0.jar`, класс `DateUtils.java`.

Требуется: разработать проект автоматизированного модульного тестирования для предоставленной библиотеки.

Задачи.

1. Создать Java-проект, подключить библиотеку `lab-1.0.jar` и зависимость JUnit5.
2. Выбрать 4 метода из предоставленного вариантом класса библиотеки.
3. Написать юнит-тесты для выбранных методов, используя фреймворк JUnit.
4. Запустить тесты через IDE.
5. Продемонстрировать результаты выполнения.

Выбранные методы для тестирования

- `isLeapYear(int)` – проверяет, является ли год високосным,
- `getDaysInMonth(int, int)` – возвращает количество дней в месяце,
- `isValidDate(String)` – проверка корректности формата даты,
- `addDays(String, int)` – добавляет указанное количество дней к дате.

3.1 Настройка проекта

Для выполнения лабораторной работы был создан Java-проект с использованием системы сборки Gradle. Файл сборки `build.gradle` содержит зависимость JUnit 5 и подключение библиотеки `lab-1.0.jar`.

В проекте реализован базовый тестовый класс `BaseTest`, содержащий аннотации жизненного цикла JUnit:

- `@BeforeAll` – инициализация перед всеми тестами,
- `@BeforeEach` – подготовка данных перед каждым тестом,
- `AfterEach` – очистка ресурсов после каждого теста,
- `@AfterAll` – завершающие операции после всех тестов.

Тестовый класс `DateUtilsTest` наследуется от `BaseTest` и содержит параметризованные тесты с использованием аннотаций `@ParameterizedTest`, `@CsvSource` и `@ValueSource` для передачи тестовых данных. В тестах проверяются классы эквивалентности и граничные значения.

3.2 Тестирование метода `isLeapYear`

Метод `isLeapYear(int year)` класса `DateUtils` проверяет, является ли год високосным. Она принимает на вход год для проверки и возвращает `true`, если год високосный, иначе возвращает `false`. Если переданное значение ≤ 0 , то функция выбрасывает `IllegalArgumentException`.

В таблице 1 приведены составленные для тестирования классы эквивалентности.

Таблица 1: Классы эквивалентности для `isYearLoop`

Входное значение	Допустимые классы	Недопустимые классы
year	кратно 400 (1), кратно 4, но не кратно 100 (2), кратно 100, но не кратно 400 (3), не кратно 4 (4)	≤ 0 (5)

Граничные значения:

- минимальное значение: `Integer.MIN_VALUE` (-2147483648);
- максимальное значение: `Integer.MAX_VALUE` (2147483647);
- нулевое значение: `year = 0`;
- минимальный корректный код: `year = 1`;
- минимальный високосный год: `year = 4`.

В таблице 2 приведены результаты тестирования.

Таблица 2: Результаты тестирования `isLeapYear`

year	Ожидаемый рез-т	Фактический рез-т	Статус теста
400	true	true	PASSED
4	true	true	PASSED
100	false	false	PASSED
1	false	false	PASSED
2147483647	false	false	PASSED
-2147483648	<code>IllegalArgumentException</code>	<code>IllegalArgumentException</code>	PASSED
0	<code>IllegalArgumentException</code>	<code>IllegalArgumentException</code>	PASSED

Все 7 тестов для `isLeapYear` были успешно пройдены.

3.3 Тестирование метода `getDaysInMonth`

Метод `getDaysInMonth(int year, int month)` класса `DateUtils` принимает на вход год и номер месяца и возвращает количество дней в этом месяце. Если в метод передано неположительное значение года или некорректное значение месяца ($\text{month} < 1$ или $\text{month} > 12$), то метод выбрасывает `IllegalArgumentException`.

В таблице 3 приведены составленные для тестирования классы эквивалентности.

Таблица 3: Классы эквивалентности для `getDaysInMonth`

Входное значение	Допустимые классы	Недопустимые классы
year	year > 0 (1), високосный (2), невисокосный (3)	year ≤ 0 (4)
month	1 ≤ month ≤ 12 (5)	month < 1 (6), month > 12 (7)

Граничные значения для (year, month):

- переход года через 0: (0,1) → (1,1);
- максимальное значение года: (`Integer.MAX_VALUE`, любой месяц);
- переход месяца через 0: (2024,0) → (2024,1);
- переход месяца через 12: (2024,12) → (2024,13);
- февраль в високосный год: (2024,2);
- февраль в невисокосный год: (2023,2);

В таблице 4 приведены результаты тестирования.

Таблица 4: Результаты тестирования `getDaysInMonth`

year	(month)	Ожидаемый рез-т	Фактический рез-т	Статус теста
2024	1	31	31	PASSED
2024	4	30	30	PASSED
2023	2	28	28	PASSED
2024	2	29	28	FAILED
1	1	31	31	PASSED
2147483647	1	31	31	PASSED
0	1	<code>IllegalArgumentException</code>	<code>IllegalArgumentException</code>	PASSED
-2147483648	1	<code>IllegalArgumentException</code>	<code>IllegalArgumentException</code>	PASSED
2024	0	<code>IllegalArgumentException</code>	<code>IllegalArgumentException</code>	PASSED
2024	13	<code>IllegalArgumentException</code>	<code>IllegalArgumentException</code>	PASSED

1 тест из 10 написанных не был пройден. Тест для февраля в високосный год не проходит проверку, так как возвращается значение для февраля в невисокосный год.

3.4 Тестирование метода isValidDate

Метод `isValidDate(String date)` класса `DateUtils` проверяет корректность формата даты. Он принимает на вход строку даты в формате `YYYY-MM-DD` и возвращает `true`, если формат корректен, и `false` в ином случае.

Таблица 5: Классы эквивалентности для `isValidDate`

Входное значение	Допустимые классы	Недопустимые классы
date	строка формата YYYY-MM-DD (1), год > 0 (2), $1 \leq$ месяц ≤ 12 (3), день в пределах дней месяца (4)	не строка формата YYYY-MM-DD (5), год ≤ 0 (6), месяц < 1 (7), месяц > 12 (8), день < 1 (9), день $>$ дней в месяце (10)

Граничные значения:

- **год:**

- минимальный корректный: 0001 (или 1),
- переход через ноль: 0000 $\rightarrow$ 0001,
- максимальное значение: 9999;

- **месяц:**

- минимальный корректный: 01,
- переход через границу: 00 $\rightarrow$ 01,
- максимальный корректный: 12,
- переход через границу: 12 $\rightarrow$ 13;

- **день:**

- минимальный корректный: 01,
- переход через границу: 00 $\rightarrow$ 01,
- максимальный корректный:
 - * для месяцев 1, 3, 5, 7, 8, 10, 12: 31,
 - * для месяцев 4, 6, 9, 11: 30,
 - * для февраля в невисокосный год: 28,
 - * для февраля в високосный год: 29;
- переход через границу: 31 (допустимый для января) $\rightarrow$ день 32.

Результаты тестирования приведены в таблице 6.

Из 17 написанных тестов не было пройдено 3 теста. Метод не учитывает действительное количество дней в указанном месяце и не учитывает високосность года.

Таблица 6: Результаты тестирования isValidDate

date	Ожидаемый рез-т	Фактический рез-т	Статус теста
'2024-01-15'	true	true	PASSED
'2024-02-29'	true	true	PASSED
'2023-02-29'	false	true	FAILED
'2024-02-30'	false	true	FAILED
'2024-04-31'	false	true	FAILED
'2024-13-01'	false	false	PASSED
'2024-00-01'	false	false	PASSED
'2024-01-00'	false	false	PASSED
'2024-01-32'	false	false	PASSED
'9999-01-30'	true	true	PASSED
0001-01-30	true	true	PASSED
'0000-01-01'	false	false	PASSED
'2024-01'	false	false	PASSED
'20244-01-01'	false	false	PASSED
'2024/01/01'	false	false	PASSED
'abcd-01-01'	false	false	PASSED
''	false	false	PASSED

3.5 Тестирование метода addDays

Метод `addDays(String date, int days)` класса `DateUtils` принимает на вход дату в формате `YYYY-MM-DD` и количество дней, и добавляет это количество дней к указанной дате.

Таблица 7: Классы эквивалентности для addDays

Входное значение	Допустимые классы	Недопустимые классы
date	строка формата YYYY-MM-DD (1), год > 0 (2), $1 \leq$ месяц ≤ 12 (3), день в пределах дней месяца (4)	не строка формата YYYY-MM-DD (5), год ≤ 0 (6), месяц < 1 (7), месяц > 12 (8), день < 1 (9), день $>$ дней в месяце (10)
days	целое число (1)	вещественное число (2)

Граничные значения:

- для параметра `date` граничные значения будут такими же, какие были описаны в пункте 3.4;
- для параметра `days`:
 - нулевое значение,
 - 365: увеличение на 1 год в невисокосный,

- -365: уменьшение на 1 год в невисокосный,
- 366: увеличение на 1 год в високосный,
- -366: уменьшение на 1 год в високосный;
- переход через границы дат:
 - добавление дней, приводящее к переходу на следующий месяц,
 - добавление дней, приводящее к переходу на следующий год,
 - добавление дней к 28, 29 февраля.

В таблице 8 приведены результаты тестирования метода addDays.

Таблица 8: Результаты тестирования addDays

date	days	Ожидаемый рез-т	Фактический рез-т	Статус теста
2024-01-15	0	24-01-15	24-01-15	PASSED
2024-01-31	1	24-02-01	24-02-01	PASSED
2024-12-31	1	2025-01-01	2025-01-01	PASSED
2024-02-28	1	2024-02-29	2024-02-29	PASSED
2023-02-28	1	2023-03-01	2023-03-01	PASSED
2024-02-29	1	2024-03-01	2024-03-01	PASSED
2024-01-01	365	2024-12-31	2024-12-31	PASSED
2024-01-01	366	2025-01-01	2025-01-01	PASSED
2024-02-01	-1	2024-01-31	2024-01-31	PASSED
2025-01-01	-1	2024-12-31	2024-12-31	PASSED
2024-03-01	-1	2024-02-29	2024-02-29	PASSED
2023-03-01	-1	2023-02-28	2023-02-28	PASSED
2024-12-31	-365	2024-01-01	2024-01-01	PASSED
2025-01-01	-366	2024-01-01	2024-01-01	PASSED
9999-12-31	1	10000-01-01	+10000-01-01	FAILED
2024-13-01	1	Exception	Exception	PASSED
2024-00-01	1	Exception	Exception	PASSED
0000-01-01	1	Exception	Exception	PASSED
abcd-01-01	1	Exception	Exception	PASSED
2024/01/01	1	Exception	Exception	PASSED
”	1	Exception	Exception	PASSED

Из 21 теста всего 1 тест не был пройден. Но в этом случае в спецификации не уточняется, как конкретно должна вести себя программа при выходе за границы допустимого диапазона годов, поэтому данную ситуацию нельзя однозначно считать ошибкой реализации.

3.6 Результаты юнит-тестирования

Всего написано 55 тестов:

- для `isLeapYear` написано 7 тестов, все успешно пройдены,
- для `getDaysInMonth` написано 10 тестов, 1 из которых не был пройден (метод возвращает 28 дней для февраля високосного года),
- для `isValidDate` написано 17 тестов, 3 из которых не пройдены (метод ошибочно считает корректными несуществующие даты),
- для `addDays` написан 21 тест, 1 из которых не пройден – при добавлении дня к 9999-12-31 ожидается переполнение или специальное поведение (выход за границы допустимого диапазона), но метод возвращает +10000-01-01. Это нельзя считать явной ошибкой, так как спецификация не описывает поведение программы при переполнении года.

Основные проблемы библиотеки связаны с некорректной обработкой граничных условий для дат (високосные годы, количество дней в месяцах). Методы `isLeapYear` и `addDays` (кроме границы 9999 год) работают корректно.

4 Задача 2. UI-тестирование веб-сайта

Дано:

- тестовый веб-сайт: <https://jdi-testing.github.io/jdi-light/index.html>,
- браузер: Google Chrome,
- фреймворки: Selenium версии 4.15 и TestNG версии 7.7,
- учетные данные для входа.

Требуется: разработать автоматизированные тесты для проверки корректности отображения и функциональности страниц сайта.

Задачи.

1. Создать проект, подключить зависимости Selenium WebDriver и TestNG.
2. Настроить класс инициализации драйвера (DriverSetup):
 - (a) Использовать аннотацию `@BeforeTest` для запуска браузера, настройки окна и перехода на сайт.
 - (b) Использовать аннотацию `@AfterTest` для корректного закрытия браузера (`driver.quit()` или `driver.close()`).
3. Разработать тестовые сценарии: создать два отдельных тестовых класса (или метода), соответствующих сценариям.

4.1 Тестовые сценарии

Первый и второй тест должны проигрывать 1 и 2 сценарий соответственно. Первый тестовый сценарий приведен в таблице 9, второй – в таблицах 10-13.

Таблица 9: Тестовый сценарий 1

№	Шаг	Данные	Ожидаемый результат
1	Перейти на страницу "Support" через главное меню	Пункт меню: "Service" → "Support"	Страница Support открыта
2	Проверить заголовок страницы	Ожидаемый текст: "support" (без учёта регистра)	Заголовок содержит слово "support"

3	Найти форму на странице	Поиск по тегу: <code>form</code>	Найдена хотя бы одна форма
4	Если форма не найдена, найти контактный блок	Поиск по селектору: <code>div[class*='contact-section'][class*='contact-block']</code>	Найден хотя бы один контактный блок
5	Проверить, что форма отображается	Проверка видимости элемента	Форма видима пользователю
6	Проверить наличие input-полей в форме	Поиск по тегу: <code>input</code> внутри формы	В форме есть хотя бы 1 input
7	Проверить наличие textarea в форме	Поиск по тегу: <code>textarea</code> внутри формы	В форме есть хотя бы 1 textarea
8	Проверить наличие кнопки отправки в форме	Поиск по селектору: <code>button, input[type='submit']</code>	В форме есть кнопка отправки

Таблица 10: Тестовый сценарий 2

№	Шаг	Данные	Ожидаемый результат
1	Получить заголовок страницы	Метод: <code>driver.getTitle()</code>	Заголовок получен
2	Проверить соответствие	Ожидаемый: <code>HomePage</code>	Заголовок совпадает
3	Проверить, что не пустой	Проверка на пустую строку	Заголовок не пустой
4	Проверить содержание	Ожидаемое: содержит <code>home</code> (без учёта регистра)	Заголовок содержит ключевое слово

Таблица 11: Тестовый сценарий 3

№	Шаг	Данные	Ожидаемый результат
1	Найти пункты верхнего меню	Селектор: <code>ul.uui-navigation.nav > li</code>	Найдено 4 элемента
2	Проверить количество	Ожидаемое: 4	Количество равно 4

Таблица 12: Тестовый сценарий 4

№	Шаг	Данные	Ожидаемый результат
1	Найти пункты верхнего меню	Селектор: <code>ul.uui-navigation.nav > li</code>	Найдено 4 элемента
2	Проверить текст #1	Ожидаемый: HOME	Текст совпадает
3	Проверить текст #2	Ожидаемый: CONTACT FORM	Текст совпадает
4	Проверить текст #3	Ожидаемый: SERVICE	Текст совпадает
5	Проверить текст #4	Ожидаемый: METALS & COLORS	Текст совпадает

Таблица 13: Тестовый сценарий 5

№	Шаг	Данные	Ожидаемый результат
1	Найти ссылки меню	Селектор: <code>ul.uui-navigation.nav > li > a</code>	Найдено 4 элемента
2	Проверить видимость и активность	Методы: <code>isDisplayed()</code> , <code>isEnabled()</code>	Все ссылки отображаются и активны
3	Проверить href #1	Ожидаемое: содержит <code>index.html</code> или <code>#</code> или <code>jdi-testing</code>	href соответствует паттерну
4	Проверить href #2	Ожидаемое: содержит <code>index.html</code> или <code>#</code> или <code>jdi-testing</code>	href соответствует паттерну
5	Проверить href #3	Ожидаемое: содержит <code>index.html</code> или <code>#</code> или <code>jdi-testing</code>	href соответствует паттерну
6	Проверить href #4	Ожидаемое: содержит <code>index.html</code> или <code>#</code> или <code>jdi-testing</code>	href соответствует паттерну

4.2 Настройка драйвера (класс DriverSetup)

Для инициализации и настройки драйвера WebDriver был создан класс DriverSetup, который выполняет функции подготовки и последующего освобождения ресурсов, необходимых для выполнения тестов. Данный класс ис-

пользует аннотации фреймворка TestNG для определения момента инициализации и завершения работы браузера.

Метод `setup()` Метод `setup()` помечен аннотацией `@BeforeTest` – он выполняется однократно перед запуском всех тестов. В теле метода происходит создание экземпляра драйвера для браузера Google Chrome. После запуска браузера драйвер переходит по URL тестового стенда с помощью метода `navigate().to()`, после чего выполняет процедуру авторизации: клик по кнопке вызова формы входа, ввод логина и пароля в соответствующие поля и нажатие кнопки подтверждения входа.

Метод `exit()` Метод `exit()` помечен аннотацией `@AfterTest` и выполняется однократно после завершения всех тестов. Он вызывает метод `close()`, который закрывает текущее окно браузера.

4.3 Класс `NavigationTrainigResult`

Класс `NavigationTrainigResult` наследуется от `DriverSetup` и реализует тестовый сценарий, проверяющий корректность навигации до страницы "Support" и структуру этой страницы.

4.3.1 Навигация к целевой странице

Класс использует `WebDriverWait` и `Actions` для взаимодействия с выпадающим меню "Service". Навигация включает следующие шаги: ожидание кликабельности пункта "Service", наведение курсора, ожидание появления целевого пункта "Support", прокрутка до него с помощью JavaScript и клик.

4.3.2 Выполняемые проверки

Проверки, выполняемые методом `testSupportPageFullCheck`:

1. **Проверка заголовка страницы.** Полученный заголовок страницы приводится к нижнему регистру и проверяется на наличие подстроки "support". При отсутствии подстроки тест фиксирует ошибку с указанием фактического значения заголовка.
2. **Проверка наличия формы.** Выполняется поиск элементов с тегом `form`. Если форма обнаружена, тест переходит к проверкам её содержания. При отсутствии формы выполняется альтернативная проверка наличия контактного блока.
3. **Проверка видимости формы.** Проверяется, что найденная форма отображается на странице (метод `isDisplayed`).

4. **Проверка наличия полей ввода.** Внутри формы выполняется поиск элементов с тегом `input`. Проверяется, что найдено как минимум одно поле ввода.
5. **Проверка наличия текстовой области.** Внутри формы выполняется поиск элементов с тегом `textarea`. Проверяется наличие как минимум одной текстовой области.
6. **Проверка наличия кнопки отправки.** Внутри формы выполняется поиск элементов по CSS-селекторам `button` и `input[type='submit']`. Проверяется, что найдена хотя бы одна кнопка.
7. **Альтернативная проверка контактного блока (выполняется, если форма не найдена).** Осуществляется поиск элементов по CSS-селекторам `div[class='contact']` и `section[class='contact']`. Проверяется, что найден хотя бы один контактный блок.

4.3.3 Отладка

Метод `saveDebugInfo` при возникновении ошибки выполняет следующие действия:

- создаёт скриншот текущего состояния страницы с помощью интерфейса `TakesScreenshot` и сохраняет его в файл с именем `error_<имя_теста>.png`,
- получает HTML-код текущей страницы через метод `getPageSource` и сохраняет его в текстовый файл с именем `error_<имя_теста>.html`,
- выводит в консоль уведомление о сохранении отладочной информации.

4.4 Класс `HomePageNavigationTestsResult`

Класс `HomePageNavigationTestsResult` наследуется от `DriverSetup` и реализует комплексную проверку главной страницы тестируемого веб-сайта. Данный класс содержит десять тестовых методов, каждый из которых выполняет проверку определённого элемента или свойства страницы.

4.4.1 Выполняемые проверки

Класс выполняет следующие проверки:

1. **Проверка заголовка страницы.** Метод `testPageTitle` получает заголовок вкладки браузера через `driver.getTitle()` и проверяет, что он равен строке "Home Page". Дополнительно проверяется, что заголовок не является пустым и содержит слово "home" (без учёта регистра).

2. **Проверка количества пунктов верхнего меню.** Метод `testHeaderMenuCount` находит все элементы `li` внутри навигационной панели `ul.uui-navigation.nav` и проверяет, что их количество равно четырём.
3. **Проверка текстов пунктов верхнего меню.** Метод `testHeaderMenuTexts` извлекает текст каждого пункта меню, удаляет пробелы и приводит к верхнему регистру, после чего сравнивает с ожидаемыми значениями: "HOME "CONTACT FORM "SERVICE "METALS & COLORS". Данный метод зависит от успешного выполнения проверки количества пунктов меню.
4. **Проверка ссылок верхнего меню.** Метод `testHeaderMenuLinks` находит все ссылки (a) внутри пунктов меню и для каждой проверяет отображение (`isDisplayed`) и активность (`isEnabled`). Если атрибут `href` присутствует, проверяется, что он содержит один из ожидаемых паттернов: "index.html "contact.html "service.html "metals.html символ "#"или строку "jdi-testing".
5. **Проверка кликабельности пунктов меню.** Метод `testHeaderMenuClickable` дополнительно убеждается, что все ссылки верхнего меню отображаются и активны, что гарантирует их доступность для взаимодействия.
6. **Проверка имени залогиненного пользователя.** Метод `testLoggedInUserName` находит элемент с идентификатором `user-name`, проверяет его видимость и извлекает текст. Проверяется, что текст не пустой, содержит "ROMAN"или "IOVLEV"(без учёта регистра) и состоит только из букв латинского алфавита и пробелов.
7. **Проверка аватара пользователя.** Метод `testUserAvatar` находит иконку профиля по селектору `.profile-photo`, `.user-icon`. Проверяется видимость и активность элемента, а также то, что его размеры (ширина и высота) больше нуля. Дополнительно проверяется наличие визуального контента: текст, класс, содержащий "icon"/"avatar"/"photo или тег `img/svg`.
8. **Проверка логотипа сайта.** Метод `testSiteLogo` выполняет поиск логотипа по нескольким селекторам (`.brand-logo`, `.logo`, `img[alt*='logo']`, `.jdi-logo`), а при отсутствии результатов — находит первый элемент `img` внутри заголовка страницы. Проверяется видимость логотипа, наличие непустого атрибута `alt`, а также то, что логотип обернут в ссылку, `href` которой ведёт на главную страницу (содержит "index "/"или "jdi-testing").

9. **Проверка футера страницы.** Метод `testFooterContent` находит подвал страницы по селекторам `footer`, `.footer` или `[role='contentinfo']`. Проверяется видимость футера и непустота его текстового содержимого. Дополнительно проверяется, что текст футера содержит хотя бы один из ожидаемых элементов: символ копирайта, слово "copyright четырёхзначный год, "jdi" или "epam".
10. **Проверка адаптивных классов.** Метод `testResponsiveClasses` находит основной контейнер страницы по селекторам `.container`, `.wrapper` или `.main-content` и проверяет, что его атрибут `class` содержит одно из ключевых слов: "container", "wrapper" или "content". Данная проверка подтверждает использование адаптивной вёрстки на сайте.

4.5 Результаты тестирования

Все тесты из `NavigationTrainigResult` и `HomePageNavigationTestsResult` были успешно пройдены. Результаты запуска через TestNG представлены на рис. 1.

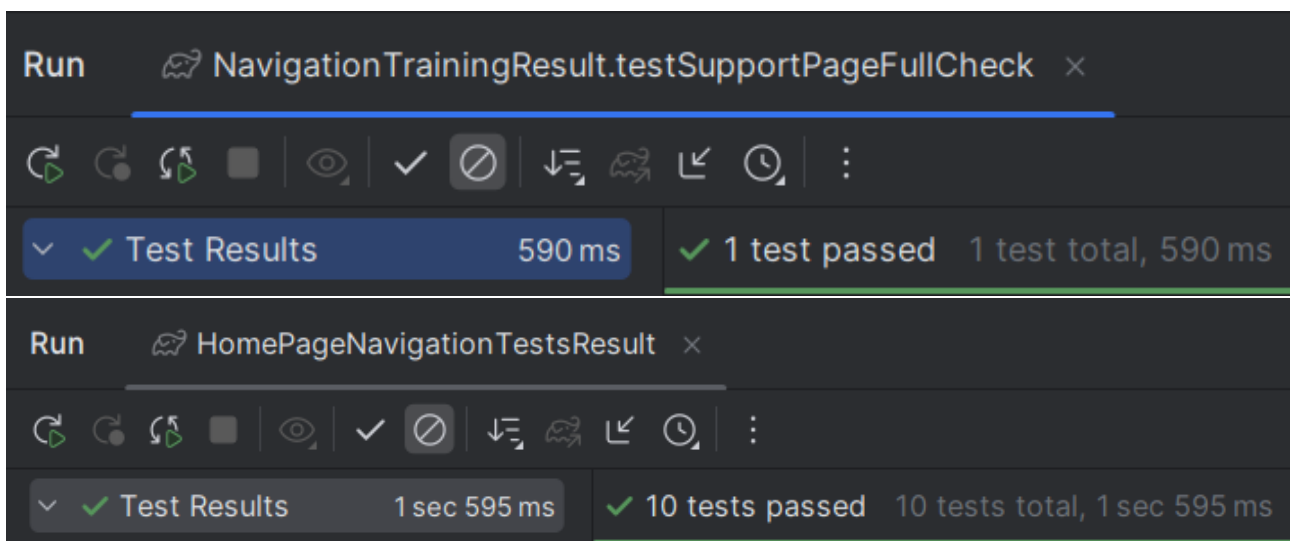


Рис. 1: Результаты тестирования

Заключение

В ходе выполнения лабораторной работы были изучены и применены на практике основные инструменты автоматизированного тестирования программного обеспечения: фреймворк JUnit для модульного тестирования и Selenium WebDriver и TestNG для UI-тестирования веб-приложений.

В рамках первой задачи было проведено модульное тестирование библиотеки `lab-1.0.jar`. Для четырёх методов класса `DateUtils` (`isLeapYear`, `getDaysInMonth`, `isValidDate`, `addDays`) были разработаны тестовые сценарии с использованием классов эквивалентности и граничных значений. Всего было написано 55 тестов. В результате тестирования выявлены следующие дефекты: метод `getDaysInMonth` некорректно обрабатывает февраль високосного года, возвращая 28 дней вместо 29; метод `isValidDate` ошибочно считает корректными несуществующие даты (например, 2023-02-29, 2024-02-30, 2024-04-31). Методы `isLeapYear` и `addDays` (за исключением случая переполнения года) отработали корректно. Таким образом, основная проблема библиотеки связана с недостаточной обработкой граничных условий при работе с датами.

Во второй задаче было проведено UI-тестирование веб-сайта с использованием Selenium WebDriver и TestNG. Был реализован класс `DriverSetup` для инициализации браузера и авторизации, а также два тестовых класса: `NavigationTrainingResult`, проверяющий навигацию до страницы "Support" и структуру формы на ней, и `HomePageNavigationTestsResult`, выполняющий комплексную проверку главной страницы (заголовок, верхнее меню, пользовательская информация, логотип, футер, адаптивные классы). Все тесты были успешно пройдены.

Список литературы

- [1] **Майерс, Г. Искусство тестирования программ** / Г. Майерс, Т. Баджетт, К. Сандлер. - Изд. 3-е. – Санкт-Петербург: Диалектика, 2012 - 272 с.

Приложение А. Исходный код DateUtilsTests

```
1  import org.junit.jupiter.api.*;
2  import org.junit.jupiter.params.ParameterizedTest;
3  import org.junit.jupiter.params.provider.CsvSource;
4  import org.junit.jupiter.params.provider.ValueSource;
5  import static org.junit.jupiter.api.Assertions.*;
6
7  abstract class BaseTest {
8
9      @BeforeAll
10     static void globalSetup() {
11         System.out.println("ЗАПУСК ТЕСТОВ DateUtils");
12     }
13
14     @BeforeEach
15     void setup() {
16         System.out.println("\n--> Подготовка к тесту");
17     }
18
19     @AfterEach
20     void cleanup() {
21         System.out.println("<-- Очистка после теста");
22     }
23
24     @AfterAll
25     static void globalTeardown() {
26
27         System.out.println("ТЕСТИРОВАНИЕ ЗАВЕРШЕНО");
28     }
29 }
30
31
32 public class DateUtilsTest extends BaseTest {
33     @ParameterizedTest
34     @CsvSource({
35         "400, true",
36         "4, true",
37         "100, false",
38         "1, false",
39         "2147483647, false"
40     })
41     void testIsLeapYear(int year, boolean expected) {
42         boolean result = ru.spbstu.hsai.DateUtils.isLeapYear(year);
43         assertEquals(expected, result, "Ошибка для года " + year);
44     }
45
46     @ParameterizedTest
47     @ValueSource(ints = {0, -2147483648, Integer.MIN_VALUE})
```

```

48 void testIsLeapYearInvalidYears(int invalidYear) {
49     assertThrows(IllegalArgumentException.class,
50         () -> ru.spbstu.hsai.DateUtils.isLeapYear(
51             invalidYear),
52         "Год " + invalidYear + " должен вызывать
53             IllegalArgumentException");
54 }
55
56 @ParameterizedTest
57 @CsvSource({
58     "2024, 1, 31",
59     "2024, 4, 30",
60     "2023, 2, 28",
61     "2024, 2, 29",
62     "1, 1, 31",
63     "2147483647, 1, 31"
64 })
65 void testGetDaysInMonth(int year, int month, int expectedDays
66 ) {
67     int result = ru.spbstu.hsai.DateUtils.getDaysInMonth(year
68         , month);
69     assertEquals(expectedDays, result,
70         String.format("Ошибка: %d-%d должен иметь %d дней
71             ", year, month, expectedDays));
72 }
73
74 @ParameterizedTest
75 @ValueSource(ints = {0, Integer.MIN_VALUE})
76 void testGetDaysInMonthInvalidYear(int invalidYear) {
77     assertThrows(IllegalArgumentException.class,
78         () -> ru.spbstu.hsai.DateUtils.getDaysInMonth(
79             invalidYear, 1),
80         "Год " + invalidYear + " должен вызывать
81             IllegalArgumentException");
82 }
83
84 @ParameterizedTest
85 @CsvSource({
86     "2024, 0", "2024, 13", "2024, -1"
87 })
88 void testGetDaysInMonthInvalidMonth(int year, int
89     invalidMonth) {
90     assertThrows(IllegalArgumentException.class,
91         () -> ru.spbstu.hsai.DateUtils.getDaysInMonth(
92             year, invalidMonth),
93         "Месяц " + invalidMonth + " должен вызывать
94             IllegalArgumentException");
95 }
96
97 @ParameterizedTest
98 @CsvSource({
99     "'2024-01-15', true",

```

```

90         " '2024-02-29', true",
91         " '2023-02-29', false",
92         " '2024-02-30', false",
93         "2024-04-31, false",
94         " '2024-13-01', false",
95         " '2024-00-01', false",
96         " '2024-01-00', false",
97         " '2024-01-32', false",
98         "9999-01-30, true",
99         "0001-01-01, true",
100        " '0000-01-01', false",
101        " '2024-01', false",
102        " '20244-01-01', false",
103        " '2024/01/01', false",
104        " 'abcd-01-01', false",
105        "'' , false",
106    })
107    void testIsValidDate(String date, boolean expected) {
108        System.out.println("Дата: '" + date + "' - корректна? Ожи
109        дается: " + expected);
110        boolean result = ru.spbstu.hsai.DateUtils.isValidDate(
111            date);
112        assertEquals(expected, result, "Ошибка валидации для даты
113            : " + date);
114    }
115
116    @Test
117    void testIsValidDateNull() {
118        System.out.println("Тест с null");
119        assertThrows(Exception.class,
120            () -> ru.spbstu.hsai.DateUtils.isValidDate(null),
121            "Метод должен выбрасывать исключение при null");
122    }
123
124    @ParameterizedTest
125    @CsvSource({
126        "2024-01-15, 0, 2024-01-15",
127        "2024-01-31, 1, 2024-02-01",
128        "2024-12-31, 1, 2025-01-01",
129        "2024-02-28, 1, 2024-02-29",
130        "2023-02-28, 1, 2023-03-01",
131        "2024-02-29, 1, 2024-03-01",
132        "2024-01-01, 365, 2024-12-31",
133        "2024-01-01, 366, 2025-01-01",
134        "2024-02-01, -1, 2024-01-31",
135        "2025-01-01, -1, 2024-12-31",
136        "2024-03-01, -1, 2024-02-29",
137        "2023-03-01, -1, 2023-02-28",
138        "2024-12-31, -365, 2024-01-01",
139        "2025-01-01, -366, 2024-01-01",
140        "9999-12-31, 1, 10000-01-01"
141    })

```

```

139     void testAddDays(String date, int days, String expected) {
140         String result = ru.spbstu.hsai.DateUtils.addDays(date,
141             days);
142         assertEquals(expected, result);
143     }
144
145     @ParameterizedTest
146     @CsvSource({
147         "2024-13-01, 1",
148         "0000-01-01, 1",
149         "abcd-01-01, 1",
150         "2024-00-01, 1",
151         "2024/01/01, 1",
152         "' ', 1"
153     })
154     void testAddDaysInvalidDate(String date, int days) {
155         assertThrows(Exception.class,
156             () -> ru.spbstu.hsai.DateUtils.addDays(date, days
157             ));
158     }
159 }

```

Листинг 1 Исходный код DateUtilsTests

Исходный код DriverSetup

```
1 package ui;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.WebDriver;
5 import org.openqa.selenium.chrome.ChromeDriver;
6 import org.testng.annotations.AfterTest;
7 import org.testng.annotations.BeforeTest;
8
9 public class DriverSetup {
10     protected static WebDriver driver;
11
12     @BeforeTest
13     public static void setup() {
14         driver = new ChromeDriver();
15
16         driver.navigate().to("https://jdi-testing.github.io/jdi-
17             light/index.html");
18
19         driver.findElement(By.cssSelector("html > body > header >
20             div > nav > ul.ui-navigation.navbar-nav.navbar-right
21             > li > a > span")).click();
22         driver.findElement(By.id("name")).sendKeys("Roman");
23         driver.findElement(By.id("password")).sendKeys("Jdi1234");
24         ;
25         driver.findElement(By.id("login-button")).click();
26     }
27
28     @AfterTest
29     public static void exit() {
30         //10. Close Browser
31         driver.close();
32     }
33 }
```

Листинг 2 Исходный код DriverSetup

Исходный код NavigationTrainigResult

```
1 package ui;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.OutputType;
5 import org.openqa.selenium.TakesScreenshot;
6 import org.openqa.selenium.WebElement;
7 import org.openqa.selenium.support.ui.ExpectedConditions;
8 import org.openqa.selenium.support.ui.WebDriverWait;
9 import org.testng.annotations.Test;
```

```

10 import org.testng.asserts.SoftAssert;
11
12 import java.io.File;
13 import java.io.FileWriter;
14 import java.io.IOException;
15 import java.time.Duration;
16 import java.util.List;
17
18 public class NavigationTrainingResult extends DriverSetup {
19
20     private void navigateToSupport() {
21         WebDriverWait wait = new WebDriverWait(driver, Duration.
            ofSeconds(15));
22
23         try {
24             WebElement aboutLink = wait.until(ExpectedConditions.
                elementToBeClickable(
25                     By.xpath("//a[contains(text(), 'About')]")
26                 ));
27             aboutLink.click();
28             Thread.sleep(1500);
29         } catch (Exception e) {
30             throw new RuntimeException("Не удалось перейти на стр
                аницу Support", e);
31         }
32     }
33
34     private void saveDebugInfo(String testName, Exception e) {
35         try {
36             File src = ((TakesScreenshot) driver).getScreenshotAs
                (OutputType.FILE);
37             src.renameTo(new File("error_" + testName + ".png"));
38
39             String html = driver.getPageSource();
40             FileWriter writer = new FileWriter("error_" +
                testName + ".html");
41             writer.write(html);
42             writer.close();
43
44             System.out.println("DEBUG: Скриншот и HTML сохранены
                для теста " + testName);
45         } catch (IOException ex) {
46             ex.printStackTrace();
47         }
48     }
49
50     @Test
51     public void testSupportPageFullCheck() {
52         SoftAssert softAssert = new SoftAssert();
53         String testName = "SupportPage";
54
55         try {

```

```

56 // 1. Navigate to the "Support" page using the main
57 // menu navigation
58 navigateToSupport();
59
60 // 2. Assert that the page title contains the word "
61 // support" (case-insensitive)
62 String title = driver.getTitle().toLowerCase();
63 softAssert.assertTrue(title.contains("support"),
64 "Page title should contain 'support'. Actual
65 title: " + title);
66
67 // 3. Assert that a form or contact block is present
68 // on the page
69 List<WebElement> forms = driver.findElements(By.
70 tagName("form"));
71 softAssert.assertFalse(forms.isEmpty(), "Form or
72 contact block should be present on the page");
73
74 if (!forms.isEmpty()) {
75     WebElement form = forms.get(0);
76
77     // 4. Assert that the form is NOT displayed and
78     // visible to the user
79     ((org.openqa.selenium.JavascriptExecutor) driver)
80     .executeScript("arguments[0].scrollIntoView(
81 true);", form);
82 Thread.sleep(500);
83 softAssert.assertFalse(form.isDisplayed(),
84 "Form should NOT be displayed and visible
85 to the user");
86
87 // 5. Assert that the form contains at least one
88 // input field (<input>)
89 List<WebElement> inputs = form.findElements(By.
90 xpath("./input[not(@type='hidden') and not(
91 @type='submit') and not(@type='button')]"));
92 softAssert.assertTrue(inputs.size() >= 1,
93 "Form should contain at least one input
94 field. Found: " + inputs.size());
95
96 // 6. Assert that the form does NOT contain a
97 // textarea element
98 List<WebElement> textAreas = form.findElements(By
99 .tagName("textarea"));
100 softAssert.assertTrue(textAreas.isEmpty(),
101 "Form should NOT contain textarea element
102 . But found: " + textAreas.size());
103
104 // 7. Assert that the form contains a submit
105 // button
106 List<WebElement> submitButtons = form.
107 findElements(By.xpath("./button[@type='submit

```



```

89         ''] | .//input[@type='submit'] | .//button[not(
90         @type)])");
        softAssert.assertTrue(!submitButton.isEmpty(),
            "Form should contain a submit button.
            Found: " + submitButtons.size());
91     }
92
93     } catch (Exception e) {
94         saveDebugInfo(testName, e);
95         softAssert.fail("Error in test " + testName + ": " +
            e.getMessage());
96     }
97
98     softAssert.assertAll();
99 }
100 }

```

Листинг 3 Исходный код NavigationTrainigResult

Исходный код HomePageNavigationTestsResult

```

1  package ui;
2
3  import org.openqa.selenium.By;
4  import org.openqa.selenium.WebElement;
5  import org.testng.annotations.Test;
6  import java.util.List;
7  import static org.testng.Assert.*;
8
9  public class HomePageNavigationTestsResult extends DriverSetup {
10
11     @Test(priority = 1)
12     public void testPageTitle() {
13         String title = driver.getTitle();
14         assertEquals(title, "Home Page",
15             "Заголовок вкладки не совпадает с ожидаемым 'Home
16             Page'. Получено: " + title);
17
18         assertFalse(title.trim().isEmpty(), "Заголовок страницы п
19             устой");
20
21         assertTrue(title.toLowerCase().contains("home"),
22             "Заголовок должен содержать слово 'home'");
23     }
24
25     @Test(priority = 3)
26     public void testHeaderMenuCount() {
27         List<WebElement> menuItems = driver.findElements(By.
28             cssSelector("ul.ui-navigation.nav > li"));
29         assertEquals(menuItems.size(), 4,

```

```

27         "Количество пунктов в верхнем меню должно быть 4.
           Найдено: " + menuItems.size());
28     }
29
30     @Test(priority = 4, dependsOnMethods = "testHeaderMenuCount")
31     public void testHeaderMenuTexts() {
32         List<WebElement> menuItems = driver.findElements(By.
           cssSelector("ul.uii-navigation.nav > li"));
33         String[] expectedTexts = {"HOME", "CONTACT FORM", "
           SERVICE", "METALS & COLORS"};
34
35         assertEquals(menuItems.size(), expectedTexts.length,
36             "Количество элементов меню не совпадает с количес
           твою ожидаемых текстов");
37
38         for (int i = 0; i < expectedTexts.length; i++) {
39             String actualText = menuItems.get(i).getText().trim()
           .toUpperCase();
40             assertEquals(actualText, expectedTexts[i],
41                 "Текст пункта меню #" + (i+1) + " неверен. Ож
           идалось: '" + expectedTexts[i] +
42                 "', получено: '" + actualText + "'");
43         }
44     }
45
46     @Test(priority = 5, dependsOnMethods = "testHeaderMenuTexts")
47     public void testHeaderMenuLinks() {
48         List<WebElement> menuLinks = driver.findElements(By.
           cssSelector("ul.uii-navigation.nav > li > a"));
49
50         assertEquals(menuLinks.size(), 4, "В верхнем меню должно
           быть 4 ссылки");
51
52         String[] expectedHrefs = {"index.html", "contact.html", "
           service.html", "metals.html"};
53
54         for (int i = 0; i < menuLinks.size(); i++) {
55             WebElement link = menuLinks.get(i);
56             String href = link.getAttribute("href");
57
58             assertTrue(link.isDisplayed(), "Ссылка меню #" + (i
           +1) + " не отображается");
59             assertTrue(link.isEnabled(), "Ссылка меню #" + (i+1)
           + " не активна");
60
61             if (href != null && !href.trim().isEmpty()) {
62                 boolean containsExpected = false;
63                 for (String expected : expectedHrefs) {
64                     if (href.toLowerCase().contains(expected.
           toLowerCase())) {
65                         containsExpected = true;
66                         break;

```

```

67         }
68     }
69     assertTrue(containsExpected || href.endsWith("#")
70         || href.contains("jdi-testing"),
71         "href пункта меню #" + (i+1) + " должен с
72         одержать один из ожидаемых паттернов:
73         " +
74         String.join(", ", expectedHrefs)
75         + ". Получено: " + href);
76     }
77 }
78
79 @Test(priority = 6, dependsOnMethods = "testHeaderMenuLinks")
80 public void testHeaderMenuClickable() {
81     List<WebElement> menuItems = driver.findElements(By.
82         cssSelector("ul.ui-navigation.nav > li > a"));
83
84     for (int i = 0; i < menuItems.size(); i++) {
85         WebElement item = menuItems.get(i);
86
87         assertTrue(item.isDisplayed(), "Пункт меню #" + (i+1)
88             + " не отображается");
89         assertTrue(item.isEnabled(), "Пункт меню #" + (i+1) +
90             " не активен");
91     }
92 }
93
94 @Test(priority = 7)
95 public void testLoggedInUserName() {
96     WebElement userNameElement = driver.findElement(By.id("
97         user-name"));
98     assertTrue(userNameElement.isDisplayed(), "Элемент с имен
99         ем пользователя не найден");
100
101     String userName = userNameElement.getText().trim();
102     assertFalse(userName.isEmpty(), "Имя пользователя пустое"
103         );
104
105     assertTrue(userName.toUpperCase().contains("ROMAN") ||
106         userName.toUpperCase().contains("IOVLEV"),
107         "Неверное имя пользователя: " + userName + ". Ожи
108         далось содержание 'ROMAN' или 'IOVLEV'");
109
110     assertTrue(userName.matches("[a-zA-Z\\s]+$"),
111         "Имя пользователя содержит недопустимые символы:
112         " + userName);
113 }
114
115 @Test(priority = 8, dependsOnMethods = "testLoggedInUserName"
116     )
117 public void testUserAvatar() {

```

```

105     WebElement userIcon = driver.findElement(By.cssSelector(".profile-photo, .user-icon"));
106
107     assertTrue(userIcon.isDisplayed(), "Иконка пользователя не отображается");
108     assertTrue(userIcon.isEnabled(), "Иконка пользователя не активна");
109
110     int width = userIcon.getSize().getWidth();
111     int height = userIcon.getSize().getHeight();
112     assertTrue(width > 0 && height > 0,
113         "Иконка пользователя имеет нулевые размеры: " +
114         width + "x" + height);
115
116     String tagName = userIcon.getTagName();
117     String innerText = userIcon.getText().trim();
118     String className = userIcon.getAttribute("class");
119
120     boolean hasVisualContent =
121         !innerText.isEmpty() ||
122         className.toLowerCase().matches(".*(icon|avatar|photo|user).*") ||
123         tagName.equals("img") || tagName.equals("svg");
124
125     assertTrue(hasVisualContent,
126         "Элемент иконки пользователя не содержит визуального контента: tag=" + tagName +
127         ", class=" + className + ", text='" + innerText + "'");
128
129     }
130
131     @Test(priority = 10)
132     public void testSiteLogo() {
133         WebElement logo = null;
134         try {
135             logo = driver.findElement(By.cssSelector(".brand-logo, .logo, img[alt*='logo'], .jdi-logo"));
136         } catch (Exception e) {
137             // Если не нашли по классам, ищем первый img в header
138             logo = driver.findElement(By.cssSelector("header img"));
139         }
140
141         assertNotNull(logo, "Логотип сайта не найден");
142         assertTrue(logo.isDisplayed(), "Логотип не отображается");
143
144         String alt = logo.getAttribute("alt");
145         assertNotNull(alt, "У логотипа отсутствует атрибут alt");

```

```

145     WebElement logoLink = logo.findElement(By.xpath("./
146         ancestor::a[1]"));
147     assertNotNull(logoLink, "Логотип не обернут в ссылку");
148
149     String href = logoLink.getAttribute("href");
150     assertNotNull(href, "У ссылки логотипа отсутствует href")
151     ;
152     assertTrue(href.contains("index") || href.endsWith("/")
153         || href.contains("jdi-testing"),
154         "Логотип должен вести на главную страницу");
155 }
156
157 @Test(priority = 11)
158 public void testFooterContent() {
159     WebElement footer = driver.findElement(By.cssSelector("
160         footer, .footer, [role='contentinfo']"));
161     assertTrue(footer.isDisplayed(), "Футер страницы не отобра
162         жается");
163
164     String footerText = footer.getText().trim();
165     assertFalse(footerText.isEmpty(), "Текст футера пустой");
166
167     boolean hasExpectedContent =
168         footerText.toLowerCase().contains("") ||
169         footerText.toLowerCase().contains("
170             copyright") ||
171         footerText.matches(".*\\d{4}.*") ||
172         footerText.toLowerCase().contains("jdi")
173         ||
174         footerText.toLowerCase().contains("epam")
175         ;
176
177     assertTrue(hasExpectedContent,
178         "Футер не содержит ожидаемого контента. Текст: '"
179         + footerText + "'");
180 }
181
182 @Test(priority = 12)
183 public void testResponsiveClasses() {
184     WebElement mainContainer = driver.findElement(By.
185         cssSelector(".container, .wrapper, .main-content"));
186     assertNotNull(mainContainer, "Отсутствует основной контей
187         нер страницы");
188
189     String containerClass = mainContainer.getAttribute("class
190         ");
191     assertTrue(containerClass.contains("container") ||
192         containerClass.contains("wrapper") ||
193         containerClass.contains("content"),
194         "Основной контейнер должен иметь класс container/
195         wrapper/content");
196 }

```

Листинг 4 Исходный код HomePageNavigationTestsResult